

Context-Free Grammars and Constituency Parsing

Thành viên thực hiện:

Hà Thanh Phong - 24520024

Hà Xuân Thiện - 24520031

Trần Quang Trường - 24521901

University of Information Technology - VNUHCM

Ngày 12 tháng 5 năm 2026

- ➊ CKY Parsing: A Dynamic Programming Approach
- ➋ Span-Based Neural Constituency Parsing
- ➌ Evaluating Parsers
- ➍ Heads and Head-Finding

- 1 CKY Parsing: A Dynamic Programming Approach
- 2 Span-Based Neural Constituency Parsing
- 3 Evaluating Parsers
- 4 Heads and Head-Finding

18.6 CKY Parsing: A Dynamic Programming Approach

Khái niệm

CKY (Cocke-Kasami-Younger) là thuật toán parsing dựa trên **quy hoạch động (dynamic programming)**.

- **Ý tưởng chính:** Thay vì phân tích lại cùng một cụm nhiều lần, ta lưu kết quả của các đoạn nhỏ vào một bảng để xây dựng các đoạn lớn hơn.

18.6.1. Conversion to Chomsky Normal Form

Để sử dụng CKY, ngữ pháp (grammar) phải được chuyển sang **Chomsky Normal Form (CNF)**.

Quy tắc Grammar trong CNF

Grammar trong CNF tuân theo một trong 2 kiểu sau:

- $A \rightarrow B C$
- $A \rightarrow w$

18.6.1. Conversion to Chomsky Normal Form

Các rule cần xử lý khi chuyển sang CNF:

- ❶ Trộn terminal và non-terminal ở vế phải.
- ❷ Unit production ($A \rightarrow B$).
- ❸ Vế phải dài hơn 2 symbol ($A \rightarrow B C D$).

18.6.1. Conversion to Chomsky Normal Form

Cách xử lý các rule:

- **Trộn Terminal và Non-terminal:**

Tạo một dummy non-terminal mới.

- Ví dụ: $\text{INF-VP} \rightarrow \text{to VP}$
- Đổi thành: $\text{INF-VP} \rightarrow \text{TO VP}$ và $\text{TO} \rightarrow \text{to}$

18.6.1. Conversion to Chomsky Normal Form

Cách xử lý các rule:

- **Trộn Terminal và Non-terminal:**

Tạo một dummy non-terminal mới.

- Ví dụ: $INF-VP \rightarrow to\ VP$
- Đổi thành: $INF-VP \rightarrow TO\ VP$ và $TO \rightarrow to$

- **Unit Production ($A \rightarrow B$):**

Loại bỏ rule trung gian, thay A bằng các rule thật mà B có thể sinh ra.

- Cách làm: Nếu có chuỗi $A \rightarrow B$ và $B \rightarrow \beta$, ta thêm quy tắc $A \rightarrow \beta$ và loại bỏ quy tắc trung gian.

18.6.1. Conversion to Chomsky Normal Form

Cách xử lý các rule:

- **Trộn Terminal và Non-terminal:**

Tạo một dummy non-terminal mới.

- *Ví dụ:* $INF-VP \rightarrow to\ VP$
- *Đổi thành:* $INF-VP \rightarrow TO\ VP$ và $TO \rightarrow to$

- **Unit Production ($A \rightarrow B$):**

Loại bỏ rule trung gian, thay A bằng các rule thật mà B có thể sinh ra.

- *Cách làm:* Nếu có chuỗi $A \rightarrow B$ và $B \rightarrow \beta$, ta thêm quy tắc $A \rightarrow \beta$ và loại bỏ quy tắc trung gian.

- **Quy tắc dài hơn 2 symbol:**

Tách thành các cặp nhị phân bằng cách sử dụng các biến trung gian ($X, Y \dots$).

- *Ví dụ:* $A \rightarrow B\ C\ D$
- *Đổi thành:* $A \rightarrow B\ X$ và $X \rightarrow C\ D$

18.6.1. Conversion to Chomsky Normal Form

$\mathcal{L}_1$ Grammar	$\mathcal{L}_1$ in CNF
$S \rightarrow NP VP$	$S \rightarrow NP VP$
$S \rightarrow Aux NP VP$	$S \rightarrow X1 VP$
	$X1 \rightarrow Aux NP$
$S \rightarrow VP$	$S \rightarrow book \mid include \mid prefer$
	$S \rightarrow Verb NP$
	$S \rightarrow X2 PP$
	$S \rightarrow Verb PP$
	$S \rightarrow VP PP$
$NP \rightarrow Pronoun$	$NP \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$NP \rightarrow United \mid Houston$
$NP \rightarrow Det Nominal$	$NP \rightarrow Det Nominal$
$Nominal \rightarrow Noun$	$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal Noun$	$Nominal \rightarrow Nominal Noun$
$Nominal \rightarrow Nominal PP$	$Nominal \rightarrow Nominal PP$
$VP \rightarrow Verb$	$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb NP$	$VP \rightarrow Verb NP$
$VP \rightarrow Verb NP PP$	$VP \rightarrow X2 PP$
	$X2 \rightarrow Verb NP$
$VP \rightarrow Verb PP$	$VP \rightarrow Verb PP$
$VP \rightarrow VP PP$	$VP \rightarrow VP PP$
$PP \rightarrow Preposition NP$	$PP \rightarrow Preposition NP$

Figure 18.10 $\mathcal{L}_1$ Grammar and its conversion to CNF. Note that although they aren't shown here, all the original lexical entries from $\mathcal{L}_1$ carry over unchanged as well.

18.6.2 CKY Recognition

Thuật toán CKY:

- Thuật toán thực hiện trên ma trận tam giác $[0 \dots n, 0 \dots n]$.
- Mỗi ô $[i, j]$ lưu các non-terminal có thể bao phủ từ vị trí i đến j .
- Để tìm ô $[i, j]$, thuật toán tìm điểm cắt k sao cho thỏa mãn:

$$A \rightarrow B C$$

Trong đó $B \in \text{ô } [i, k]$ (bên trái) và $C \in \text{ô } [k, j]$ (bên dưới).

- Một câu được Grammar chấp nhận khi ô $[0, n]$ chứa S .

18.6.2 CKY Recognition

```
function CKY-PARSE(words, grammar) returns table  
  
  for  $j \leftarrow$  from 1 to LENGTH(words) do  
    for all  $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$   
       $\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$   
    for  $i \leftarrow$  from  $j-2$  down to 0 do  
      for  $k \leftarrow i+1$  to  $j-1$  do  
        for all  $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C \in \text{table}[k, j]\}$   
           $\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$ 
```

Figure 18.12 The CKY algorithm.

18.6.2 CKY Recognition

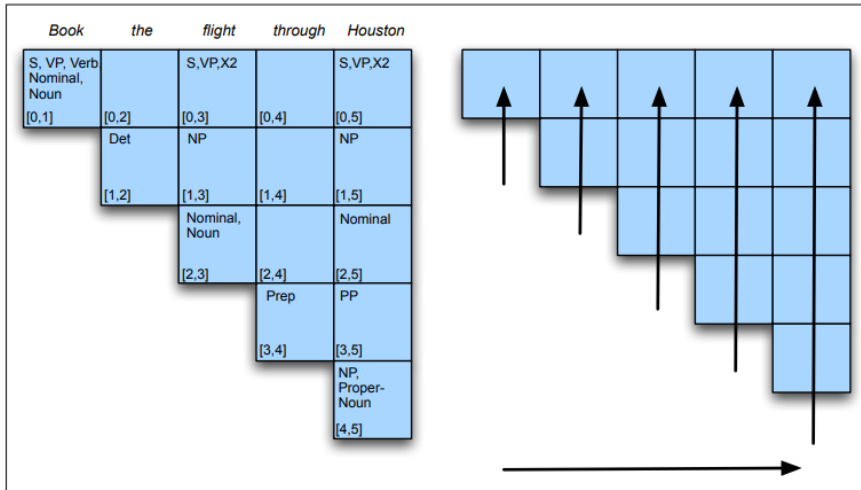


Figure 18.11 Completed parse table for *Book the flight through Houston*.

18.6.3 CKY Parsing

Thuật toán vừa đề cập chỉ cho ta biết câu được cho có parse được hay không, chứ chưa trả về Parse tree.

Để trả về Parse tree, ta cần:

- **Bổ sung Backpointers:** Mỗi non-terminal trong bảng được lưu kèm với các con trỏ dẫn đến các mục (entries) đã tạo ra nó.
- **Chấp nhận đa phiên bản:** Cho phép lưu trữ nhiều phiên bản của cùng một non-terminal trong cùng một ô nếu chúng có nguồn gốc dẫn xuất khác nhau.

Quá trình tạo Parse Tree:

- Bắt đầu từ nhãn S tại ô $[0, n]$, ta **đệ quy** xuống các thành phần con thông qua các con trỏ để dựng lại toàn bộ cây.
- Bảng CKY lúc này chứa đựng **tất cả** các cách phân tích khả dĩ cho câu đầu vào.
- Tuy nhiên, trên thực tế, thay vì lấy **mọi cây**, ta cần tìm cây "tốt nhất" (Best Parse) thông qua các mô hình xác suất.

18.6.3 CKY Parsing

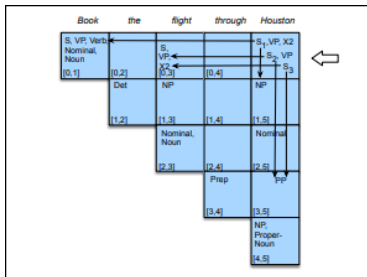
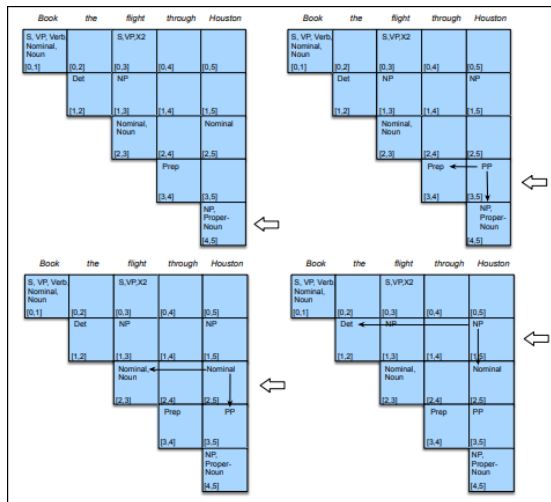


Figure 18.14. Filling the cells of column 5 after reading the word Houston



Trên thực tế, CNF có thể làm parse tree khác với cây gốc mà người thiết kế grammar mong muốn.

Ví dụ:

- Khi một rule dài bị tách thành nhiều rule nhị phân, cây parse sẽ xuất hiện thêm các **dummy non-terminal**.
- Những node giả này có ích cho thuật toán, nhưng có thể **gây khó khăn** nếu ta muốn dùng parse tree cho semantic analysis.

Khôi phục parse tree gốc (Post-processing):

- **Xử lý nút giả:**

Bằng cách xóa các nút trung gian và đẩy các con trực tiếp của nó lên lại nút cha ban đầu.

- **Xử lý Unit Productions:**

Chỉnh sửa trực tiếp thuật toán CKY để xử lý unit production trực tiếp thay vì loại bỏ hoàn toàn.

- 1 CKY Parsing: A Dynamic Programming Approach
- 2 Span-Based Neural Constituency Parsing
- 3 Evaluating Parsers
- 4 Heads and Head-Finding

18.7 Span-Based Neural Constituency Parsing

Vấn đề của CKY truyền thống

CKY liệt kê được mọi cây parse nhưng **thiếu khả năng** để chọn ra cây đúng nhất.

Ý tưởng của **Span-Based Neural Parsing** (hay Neural CKY):

- Thay vì rule thủ công, ta dùng một **Neural Classifier** để gán điểm số (*score*) cho từng span có nhãn.
- Sử dụng biến thể CKY để tìm cây có **tổng score cao nhất**.

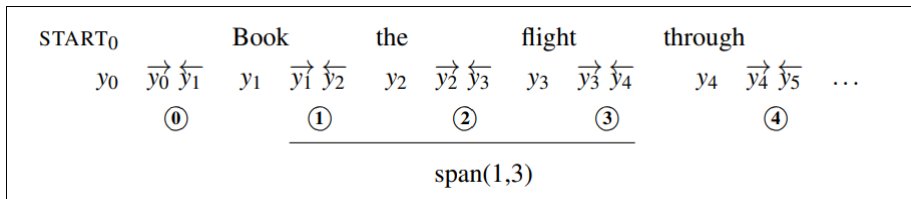
18.7.1 Computing Scores for a Span

$scores(i, j, l)$ là điểm cho giả thuyết rằng đoạn từ i đến j là một constituent loại l .

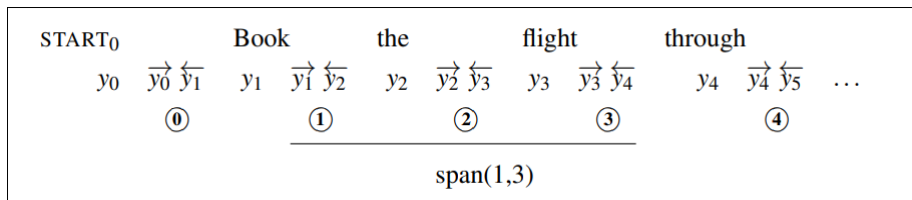
Để tính $scores(i, j, l)$, ta thực hiện các bước sau:

- Đưa câu vào mô hình ngôn ngữ.
- Đầu ra của mô hình cần được đưa về **word representation**.
- Các **word representation** này có thể được đưa qua thêm một số postprocessing layer.
- Sau đó, mô hình tạo **representation** cho từng span dựa trên hai đầu mút của span.

18.7.1 Computing Scores for a Span



18.7.1 Computing Scores for a Span



Span representation thường được tạo bằng cách lấy hiệu giữa embedding ở điểm kết thúc và điểm bắt đầu:

$$v(i, j) = [\vec{y_j} - \vec{y_i} ; \overleftarrow{y_{j+1}} - \overleftarrow{y_{i+1}}] \quad (18.5)$$

18.7.1 Computing Scores for a Span

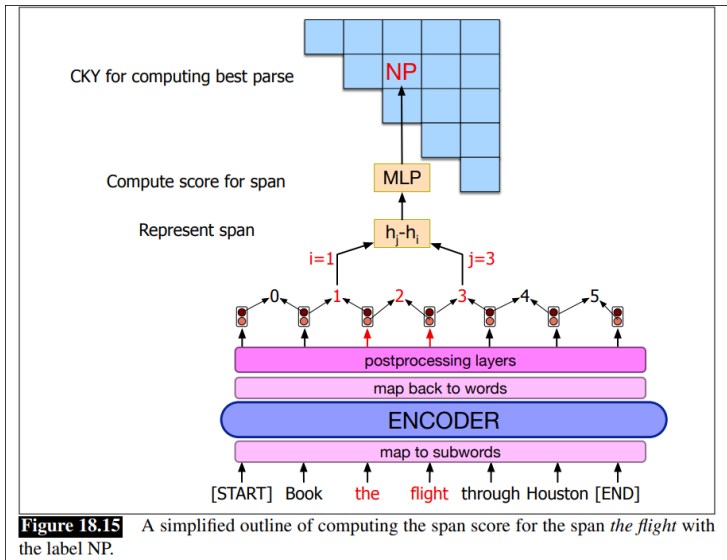
Span representation thường được tạo bằng cách lấy hiệu giữa embedding ở điểm kết thúc và điểm bắt đầu:

$$v(i, j) = [\vec{y}_j - \vec{y}_i ; \overleftarrow{\vec{y}_{j+1}} - \overleftarrow{\vec{y}_{i+1}}] \quad (18.5)$$

Sau khi có vector **span representation**, mô hình đưa vector này qua một MLP để sinh ra score cho từng non-terminal label có thể có.

$$s(i, j, \cdot) = \mathbf{W}_2 \text{ReLU}(\text{LayerNorm}(\mathbf{W}_1 \mathbf{v}(i, j))) \quad (18.6)$$

18.7.1 Computing Scores for a Span



18.7.2 Integrating Span Scores into a Parse

Gọi cây T là tập gồm $|T|$ span, với span thứ t^{th} bắt đầu từ i_t kết thúc tại j_t , với loại l_t :

$$T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\} \quad (18.7)$$

18.7.2 Integrating Span Scores into a Parse

Gọi cây T là tập gồm $|T|$ span, với span thứ t^{th} bắt đầu từ i_t kết thúc tại j_t , với loại l_t :

$$T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\} \quad (18.7)$$

Score của cả cây $s(T)$ bằng tổng score của tất cả span trong cây:

$$s(T) = \sum_{(i,j,l) \in T} s(i, j, l) \quad (18.8)$$

18.7.2 Integrating Span Scores into a Parse

Gọi cây T là tập gồm $|T|$ span, với span thứ t^{th} bắt đầu từ i_t kết thúc tại j_t , với loại l_t :

$$T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\} \quad (18.7)$$

Score của cả cây $s(T)$ bằng tổng score của tất cả span trong cây:

$$s(T) = \sum_{(i,j,l) \in T} s(i, j, l) \quad (18.8)$$

Cuối cùng, ta chọn parse tree với điểm số lớn nhất:

$$\hat{T} = \operatorname{argmax}_T s(T) \quad (18.9)$$

18.7.2 Integrating Span Scores into a Parse

Cách tiếp cận: Lựa chọn nhãn có score cao nhất $s(i, j, l)$ cho từng span.

→ Cách tiếp cận này **không đảm bảo** các span được chọn sẽ tạo thành một cây hợp lệ.

→ Trong thực tế, người ta sử dụng một biến thể của CKY để tìm cây có **tổng score tốt nhất**.

18.7.2 Integrating Span Scores into a Parse

Gọi $s_{best}(i, j)$ là điểm của subtree span (i, j) tốt nhất.

Với span có độ dài bằng 1:

$$s_{best}(i, i + 1) = \max_l s(i, i + 1, l) \quad (18.10)$$

Với span còn lại, ta gọi đệ quy:

$$\begin{aligned} s_{best}(i, j) = \max_l s(i, j, l) \\ + \max_k [s_{best}(i, k) + s_{best}(k, j)] \end{aligned} \quad (18.11)$$

- 1 CKY Parsing: A Dynamic Programming Approach
- 2 Span-Based Neural Constituency Parsing
- 3 Evaluating Parsers
- 4 Heads and Head-Finding

18.8 Evaluating Parsers

- Parser thường được đánh giá bằng PARSEVAL metrics.
- Ý tưởng của PARSEVAL là so sánh parse tree do model dự đoán với gold parse tree.
- Một **constituent** trong hypothesis parse **được xem là đúng** nếu nó khớp với constituent trong reference parse ở ba điểm:
 - Cùng vị trí bắt đầu.
 - Cùng vị trí kết thúc.
 - Cùng non-terminal label.

18.8 Evaluating Parsers

Từ đó, ta có hai metric chính:

- labeled recall = $\frac{\# \text{ of correct constituents in hypothesis parse of } s}{\# \text{ of total constituents in reference parse of } s}$
- labeled precision = $\frac{\# \text{ of correct constituents in hypothesis parse of } s}{\# \text{ of total constituents in hypothesis parse of } s}$

18.8 Evaluating Parsers

Từ đó, ta có hai metric chính:

- labeled recall = $\frac{\# \text{ of correct constituents in hypothesis parse of } s}{\# \text{ of total constituents in reference parse of } s}$
- labeled precision = $\frac{\# \text{ of correct constituents in hypothesis parse of } s}{\# \text{ of total constituents in hypothesis parse of } s}$

Ta có thể gộp cả precision và recall thành F1:

$$F1 = \frac{2PR}{P + R} \quad (18.12)$$

- 1 CKY Parsing: A Dynamic Programming Approach
- 2 Span-Based Neural Constituency Parsing
- 3 Evaluating Parsers
- 4 Heads and Head-Finding

18.9 Heads and Head-Finding

Head

Mỗi thành phần cú pháp (constituent) đều gắn liền với một từ quan trọng nhất về mặt ngữ pháp, gọi là **head**.

- Danh từ (N) là head của cụm danh từ (NP).
- Động từ (V) là head của cụm động từ (VP).

Head rất quan trọng vì nó giúp liên kết **constituency parsing** với **dependency parsing**.

18.9 Heads and Head-Finding

Head

Mỗi thành phần cú pháp (constituent) đều gắn liền với một từ quan trọng nhất về mặt ngữ pháp, gọi là **head**.

- Danh từ (N) là head của cụm danh từ (NP).
- Động từ (V) là head của cụm động từ (VP).

Head rất quan trọng vì nó giúp liên kết **constituency parsing** với **dependency parsing**.

Head-Finding

Nếu **constituency parser** cho ta cấu trúc phrase, thì head-finding giúp xác định **từ trung tâm** của mỗi phrase, từ đó có thể chuyển sang **quan hệ phụ thuộc** giữa các từ.

18.9 Heads and Head-Finding

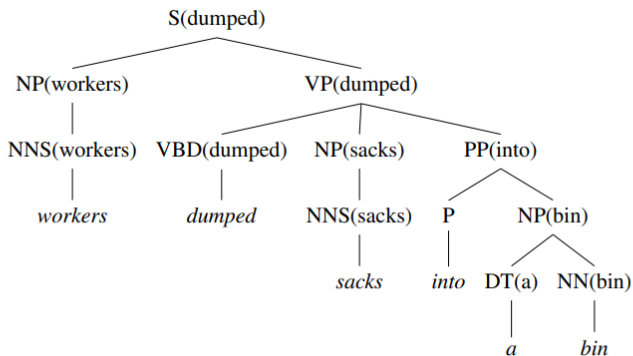


Figure 18.16 A lexicalized tree from [Collins \(1999\)](#).

18.9 Heads and Head-Finding

- Trong thực tế, **việc chọn head** không phải lúc nào cũng rõ ràng.
- Vì vậy, nhiều hệ thống dùng **head rules viết tay**.
- Các rule này duyệt qua con của một node theo **thứ tự ưu tiên** để tìm head.

18.9 Heads and Head-Finding

Ví dụ với NP, hệ thống có thể tìm từ phải sang trái để xem có noun tag như NN, NNP, NNPS hay không; nếu có thì chọn nó làm head.

- If the last word is tagged POS, return last-word.
- Else search from right to left for the first child which is an NN, NNP, NNPS, NX, POS, or JJR.
- Else search from left to right for the first child which is an NP.
- Else search from right to left for the first child which is a \$, ADJP, or PRN.
- Else search from right to left for the first child which is a CD.
- Else search from right to left for the first child which is a JJ, JJS, RB or QP.
- Else return the last word

18.9 Heads and Head-Finding

Ví dụ với NP, hệ thống có thể tìm từ phải sang trái để xem có noun tag như NN, NNP, NNPS hay không; nếu có thì chọn nó làm head.

- If the last word is tagged POS, return last-word.
- Else search from right to left for the first child which is an NN, NNP, NNPS, NX, POS, or JJR.
- Else search from left to right for the first child which is an NP.
- Else search from right to left for the first child which is a \$, ADJP, or PRN.
- Else search from right to left for the first child which is a CD.
- Else search from right to left for the first child which is a JJ, JJS, RB or QP.
- Else return the last word

→ Các head rules như vậy còn được gọi là **head percolation table**, vì head được truyền dần từ các node con lên node cha trong parse tree.

18.9 Heads and Head-Finding

Parent	Direction	Priority List
ADJP	Left	NNS QP NN \$ ADVP JJ VBN VBG ADJP JJR NP JJS DT FW RBR RBS SBAR RB
ADVP	Right	RB RBR RBS FW ADVP TO CD JJR JJ IN NP JJS NN
PRN	Left	
PRT	Right	RP
QP	Left	\$ IN NNS NN JJ RB DT CD NCD QP JJR JJS
S	Left	TO IN VP S SBAR ADJP UCP NP
SBAR	Left	WHNP WHPP WHADVP WHADJP IN DT S SQ SINV SBAR FRAG
VP	Left	TO VBD VBN MD VBZ VB VBG VBP VP ADJP NN NNS NP

Figure 18.17 Some head rules from Collins (1999). The head rules are also called a **head percolation table**.